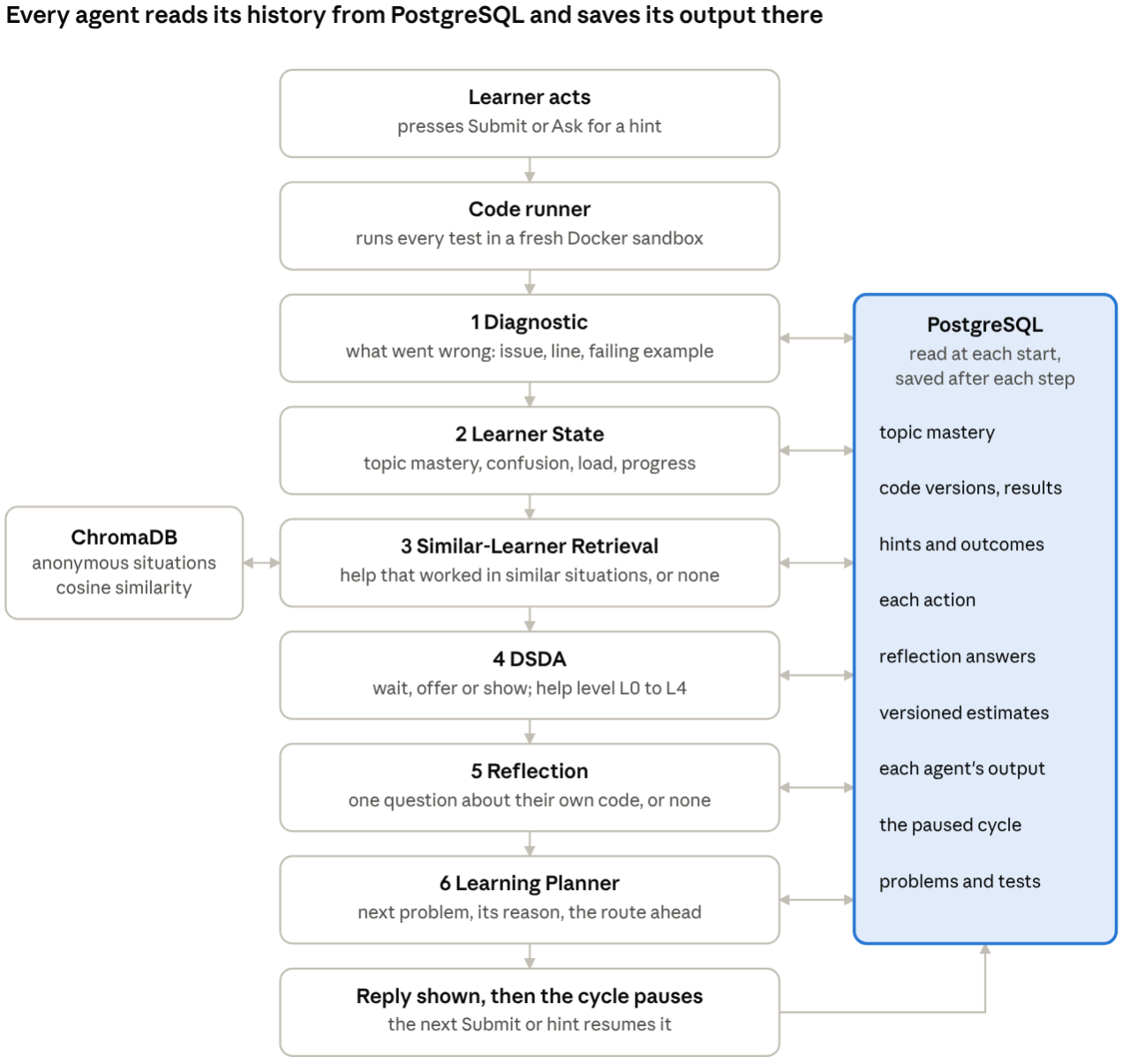
Slides 9–14 Presenter Prep Guide

Nov 11, 2026 · @lokesh

You present slides 9 to 14: the architecture, the six-agent cycle, how each agent works, the DSDA algorithm, why nothing is trained, and how we evaluated. Each slide section gives what to say, what everything on the slide means, and the facts behind it; the question sections after that give the questions to expect and how to answer them. Every fact here was checked against the project code.

The flow at a glance



one Submit or hint request · six agents, two stores

Each agent reads the learner's history from PostgreSQL at the start of the cycle and saves its output after its step; only Retrieval also uses ChromaDB. The pause at the end is saved too, so the learner's next action resumes the same cycle. Pressing Run only executes the visible examples and starts none of this.

Slide 9 · System Architecture

The point: CodePath is a web app whose backend runs six agents in a LangGraph cycle. History lives in PostgreSQL, similar situations in ChromaDB, and student code runs only in a separate Docker sandbox.

What to say (about one minute)

"This is how CodePath is built. On the left, the learner or teacher uses our web application, built with React and TypeScript: a code editor, the tutor pane, lessons, practice, progress and a teacher dashboard. It talks to our FastAPI backend, which handles logins and requests. Each learner's actions are handled one at a time, in order. When the learner submits code or asks for a hint, the LangGraph learning cycle starts, and our six agents run; I'll show them on the next slide.

The agents keep the learner's history and progress in PostgreSQL, and ChromaDB finds similar past situations. Student code never runs inside our server: it goes to a separate code runner, which starts a locked-down Docker sandbox for every run. The language model at the bottom is optional. It can only reword a hint, and it is off by default, so the tutor works fully without any paid AI model."

Every box on the slide

Box	Built with	What it does	Fact to remember
Web application	React 18, TypeScript, Vite, CodeMirror 6 editor	Learner screens (editor, tutor pane, lessons, practice, progress, profile) and the teacher's class dashboard	Runs in the browser; nothing to install
FastAPI backend	Python 3.12, FastAPI	Logins, request checking, rate limits; the API the web app calls	Every request is checked before anything runs

Box	Built with	What it does	Fact to remember
Tutoring service	Python, inside the backend	A queue per learner, so actions apply in the order they were made; a repeated click is not processed twice	Run, Submit and Hint are handled differently (table below)
LangGraph learning cycle	LangGraph	Runs the six agents in a fixed order, then pauses until the learner acts again	The pause is saved in PostgreSQL by LangGraph's checkpointer
PostgreSQL 16	Relational database	All history: code versions, results, hints, learner estimates, reflections, plans	25 tables
ChromaDB	Vector database	Stores anonymous "situation" vectors and finds the closest ones by cosine similarity	No names, no code, no problem text
Code runner	Separate service + Docker	Runs student code in a fresh container for every run	No network, read-only, 256 MB memory, 1 CPU
Language model (optional)	OpenAI, Anthropic or Ollama, only if configured	Rewords a hint the rules already chose	Off by default; it cannot change any decision

The arrows: web app and backend talk both ways (request, then the reply). The backend passes work down to the tutoring service, which starts the cycle. The cycle writes to PostgreSQL and asks ChromaDB for similar situations. The tutoring service sends student code to the code runner and gets results back. The dashed arrow to the language model means optional.

Run, Submit and Hint are not the same

This is the most important detail on this slide, because examiners often ask "when do the agents run?"

Learner presses	What happens	Agents run?	Changes mastery?
Run	Code runs on the visible examples, or the learner's own input	No	No
Submit	Code runs on all tests, visible and hidden; it counts as a graded attempt	Yes, the full cycle	Yes
Ask for a hint (or a question, or selected lines)	Code runs on the visible examples as a probe, then the cycle runs	Yes, the full cycle	No

What happens on one Submit, step by step

1. The code is saved as a new version (a code snapshot) and placed in that learner's queue.
2. The code runner runs it against every test in a fresh Docker container and returns pass or fail for each.
3. The result is stored, and the paused LangGraph cycle resumes with this new event.
4. The six agents run in order and each one saves its output.
5. The reply (result, diagnosis, maybe a hint, maybe a question, next step) appears in the tutor pane.
6. The cycle pauses again, saved in PostgreSQL, until the learner's next Submit or hint request.

Background you should know

- **Why a separate code runner:** student code is untrusted. It can loop forever, use all memory, read files or try the network. Inside the server it would put every learner at risk, and a timeout alone is not a sandbox.
- **What the sandbox blocks:** no network, read-only file system, 256 MB memory, 1 CPU, at most 64 processes, all Linux privileges dropped, runs as a nobody user, and a time limit per test. Each run gets a new container that is deleted afterwards.
- **No unsafe fallback:** if Docker is not running, the code is not run at all. The tutor says the code could not be checked; it never silently runs it unprotected.
- **Why two databases:** PostgreSQL keeps exact records of what happened. ChromaDB is built to find the nearest vectors quickly. Retrieval uses both: ChromaDB finds similar situations, PostgreSQL says what happened after the help.

- **Why a queue per learner:** if a learner clicks Submit twice quickly, the second result must not be applied before the first, or the learner's estimates would be updated out of order.

Questions on this slide

Why FastAPI? The agents and the code analysis are in Python, so a Python API keeps one language. FastAPI also checks every request against typed models automatically.

Why not run the code in the browser? Grading uses hidden tests. If grading ran in the browser, the hidden tests would have to be sent to the browser, where a learner could read them.

What happens if the sandbox is down? The submission is reported as not checked, the agents are skipped for that action, and nothing runs unprotected.

Does it scale? Today it runs on one laptop: the runner takes 3 jobs at a time with 12 queued. Scaling means more runner machines and moving the queues and rate limits to a shared store, which is in our future work.

Slide 10 · The Six-Agent Learning Cycle

The point: after every Submit or hint request, six agents run in a fixed order, each answering one question; then the tutor waits for the learner. The bottom row is our four novelty claims.

What to say (about one minute)

"This is the heart of the project. Every time the learner submits code or asks for a hint, six agents run in order. One, the Diagnostic agent finds what went wrong, with evidence. Two, the Learner State agent updates how much the learner knows. Three, Retrieval looks at what helped learners in similar situations. Four, DSDA decides whether to help and how much, and the hint is checked before it is shown. Five, Reflection may ask the learner to explain their thinking. Six, the Planner chooses what to practise next.

Then the cycle pauses until the learner acts again. Nothing runs on its own, just like a teacher who waits for the student. At the bottom are our four novel contributions: the DSDA algorithm, hybrid experience retrieval, adaptive difficulty progression, and the cyclic agent workflow that checks every hint."

The six agents in one table

#	Agent	The question it answers	What it passes on
1	Diagnostic	What went wrong, and where?	Issue label (for example off-by-one), the line, the failing example, evidence
2	Learner State	How much does the learner know now, and how are they doing?	Topic mastery with a range; confusion, load, progress, or "unknown"
3	Similar-Learner Retrieval	What helped other learners in this situation?	A suggested help level and kind of hint, or "no reliable match"
4	Adaptive Scaffolding (DSDA)	Should we help now, and how much?	Wait, offer or show; level L0–L4; the kind of hint; a reason
5	Reflection	Should we ask the learner to explain?	One short question, or nothing
6	Learning Planner	What should the learner practise next?	The next problem, its reason, and the route ahead

Why this order: each agent needs the one before it. You cannot judge the learner's state before you know what went wrong. You cannot decide how much help to give before you know the state and what helped others. You plan the next problem last, once you know how this attempt went.

What "cycle" means here

The agents are nodes in a LangGraph graph with two loops.

1. **The learning loop.** After the Planner, the graph reaches a waiting step and stops. That pause is saved in PostgreSQL, so even a server restart does not lose it. The learner's next Submit or hint request resumes the graph from the top.
2. **The checking loop.** Inside step 4, every hint goes through a validator. A rejected model draft is rewritten at the same level, with at most three drafts in all. If it still fails, the built-in hint is used; if even that cannot pass, the next unused kind of hint is tried, and if nothing passes, the tutor waits rather than show a bad hint. The checking loop never gives more detailed help.

The graph also skips stages that would be pointless: if the code could not be run, the agents are skipped; if the problem is solved, retrieval is skipped; if DSDA decides to wait or only offer, no hint is written.

The four novelty points, explained

Novelty	What it means in plain words	Why it is new (link to the literature)
Dynamic Scaffolding Decision Algorithm	Turns everything known about the learner into one decision: wait, offer or show, at which level, with which kind of hint	Knowledge tracing only predicts what a learner knows; it does not decide how much help to give
Hybrid experience retrieval	Finds past situations like this one and checks which help led to progress	RAGMan retrieves course material; we retrieve outcomes of past help. "Hybrid" = vector search in ChromaDB + outcome records in PostgreSQL
Adaptive difficulty progression	Practice goes technique by technique: easy, then medium, then hard, then the next topic	Tutors like CodeHelp and CodeAid answer questions but do not plan a learner's practice
Cyclic graph orchestration	The agents run as a graph that waits for the learner, and every hint is checked before it is shown	Single-chatbot tutors answer whenever asked; ours pauses, remembers and validates

Questions on this slide

Are these real AI agents or just functions? Each is a separate component with a typed input and output contract, its own rules and its own saved output. They are rule-based agents: they perceive the situation, reason with rules, and act. They do not need a language model.

What does LangGraph give you that plain Python would not? A graph with branches and loops, a saved pause between learner actions (the checkpointer), and a record of every step. The cycle survives a restart, and we can show exactly which agent decided what.

Why does the tutor wait instead of helping automatically? Unasked help takes the thinking away from the learner. The first nudge on a new mistake can come by itself; after that, more help is only offered, and shown when the learner asks.

What if one agent fails? Each agent has a safe fallback. For example, if Reflection fails, it is skipped; if retrieval is unavailable, DSDA decides without it. Tutoring never stops because one part failed.

Slide 11 · How Each Agent Works

The point: every agent takes a clear input, applies written rules, and gives a clear output. Know one example per agent; the running example is "Sum from 1 to N", where the learner wrote `range(1, n)`, so `sum_to_n(3)` returns 3 instead of 6.

What to say (about one and a half minutes)

"Here is how each agent works: input, rule, output. The Diagnostic agent reads the test results, the code's structure and the error message, and matches them to known beginner mistakes. Here, it finds that the loop stops one value early on line 3. The Learner State agent uses knowledge tracing, the method from our literature review: each graded attempt updates how well the learner knows the topic. It also looks for signs of confusion, and says unknown when there is no evidence.

Retrieval compares the situation with past ones and answers only when there are enough close matches. DSDA decides how much help is needed, picks a level and a new kind of hint, decides the timing, and the hint is checked before it is shown. Reflection builds a question from the learner's own code and replies to their answer. The Planner moves technique by technique, from easy to hard, then to the next topic."

1 · Diagnostic agent

- **Input:** the test results from the sandbox (which tests failed, the error type and message), the learner's code, and their previous attempt.
- **Step 1, sort the result:** solved, syntax error, runtime error (such as NameError or TypeError), wrong answer, partial (some tests pass), timeout, memory or output limit, or missing function.
- **Step 2, read the code:** Python's `ast` module turns the code into a syntax tree, which rules check for known mistakes: off-by-one, missing return, print instead of return, return inside a loop, returning nothing on some path, edge case, wrong order, wrong return type, text format, not yet written.
- **Step 3, compare with the last attempt:** which lines changed, and whether the same issue is repeating.
- **Output:** the issue label, the line, that line quoted from the learner's code, a failing visible example, and the evidence.
- **Rule it never breaks:** facts come only from evidence. Hidden test contents never reach the learner or a language model.
- **Example:** wrong answer, off-by-one on line 3; `sum_to_n(3)` gave 3, expected 6.

2 · Learner State agent

- **Input:** this attempt's result and issue; the learner's saved topic mastery; earlier attempts on this problem; hints used; time spent; their own ratings (confidence, "stuck", "overwhelmed"); reflection answers; hint feedback.
- **Mastery (knowledge tracing):** one number per topic, between 0 and 1, updated after each Submit. How much an attempt counts depends on how it went: solving alone counts fully; solving with bigger hints counts less; a syntax slip counts very little; resubmitting identical code counts nothing. A hint request never changes mastery.
- **The range:** mastery is shown with a range that narrows as evidence grows; with too little evidence it says "not enough evidence" instead of a level.
- **Confidence:** only from the learner's own rating. No rating means unknown.
- **Confusion:** needs signals such as the same issue twice in a row, a hint not followed by progress, the error type keeps changing, "I'm stuck", or a "not sure" reflection answer. Two signals = high, one = moderate. A single first failure is not confusion: it stays unknown.
- **Load:** signals such as working much longer than typical for the difficulty, three or more hints, several attempts passing no tests, or "this feels like a lot".
- **Progress:** improving, steady, stalled, regressing or solved, from the trend in tests passed.
- **Output:** each estimate with its evidence sentences, saved as a new version.
- **Say this clearly:** these are estimates from coding behaviour, with no medical or psychological meaning.

3 · Similar-Learner Retrieval agent

- **Input:** the situation as a list of 14 numbers: mastery, confidence, confusion, load, progress, difficulty, attempts and hints (8 numbers), plus 6 that mark the error category.
- **How it works:** ChromaDB finds the most similar past situations by cosine similarity. Then strict filters: similar enough, not this learner's own, outcome known, similar difficulty, same issue or same topic. It needs at least 5 such cases.
- **Choosing:** the cases are grouped by the help they received. Kinds of hint this learner has already seen are left out. The group with the best cautious success rate wins, and it needs at least 3 cases.
- **Output:** a suggested level and kind of hint, with its success rate, number of cases and where the cases came from; or "no reliable match" with the reason.
- **How DSDA uses it:** only as a nudge, at most one level, and only within DSDA's safety limits.

- **Privacy:** a case stores no name, no code and no problem text.

4 · DSDA (details in the slide 12 section)

- **Input:** everything above, plus every hint already given on this issue.
- **Output:** wait, offer or show; the level L0–L4; the kind of hint; the timing; a reason the learner can read; limits on length and on code.

5 · Reflection agent

- **When it asks:** after a solve, or after at least 3 failed graded attempts. Never when the learner seems overloaded, never on a syntax error, never while an earlier question is unanswered, and, after failed attempts, not again until at least 3 more events have passed.
- **The question:** built from the learner's own code, for example asking them to trace their loop by hand for a small input.
- **The reply:** it reads the answer for key ideas and reasoning words. It confirms a good answer, helps make a vague one concrete, and offers one small next thought if the learner is unsure. A "not sure" answer also becomes a confusion signal for the Learner State agent.
- **Never:** grades the answer or gives the fix.

6 · Learning Planner

- **Input:** topic mastery, how many problems were solved alone or with help, which techniques were practised, difficulty, and prerequisites.
- **Order inside a topic:** each technique in the lesson's order at easy, then each at medium, then hard. A technique counts as practised after 2 solves, or 1 if solved first try without help.
- **After a heavily helped solve:** one more problem with the same technique and difficulty, this time without hints.
- **Moving on:** a topic is secure only when mastery is at least 0.75, with at least 2 solves without help and enough evidence. Then the next topic forward. A weak prerequisite is suggested, not forced.
- **Output:** the next problem, why it was chosen, and the route ahead.

Questions on this slide

What is knowledge tracing? A standard method from 1995 (Corbett and Anderson, on our literature slide). It keeps a probability that the learner knows a skill and updates it after each answer, allowing for lucky guesses and careless slips.

How do you detect the error type? From the sandbox result first (did it crash, time out, or give a wrong answer), then from rules that read the code's syntax tree. No model guesses it.

What is cosine similarity? A measure of how alike two lists of numbers are, by the angle between them: 1 means the same direction. We centre each number around zero first, so a weak and a strong learner do not look alike.

What if two agents disagree? They do not vote. Each answers a different question, and DSDA makes the one decision, with fixed limits that retrieval cannot override.

Slide 12 · Dynamic Scaffolding Decision Algorithm (DSDA)

The point: DSDA decides whether, when and how much to help, in four steps: score, level, kind of hint, timing. Then the hint is written and checked. It is our main algorithm and our main novelty.

What to say (about one and a half minutes)

"DSDA is our main algorithm. Step 1 is a support score that combines what we know about the learner: how well they know the topic, how confused they are, repeated mistakes, hints that did not help, and time. Anything unknown is left out, not guessed. Step 2 turns the score into a help level from L0 to L4: a higher score means more help. But a new mistake starts at L2 or lower, and help rises only one level at a time.

Step 3 picks the next unused kind of hint for this mistake, so a hint never repeats. Step 4 is timing. If the learner asked, we show the hint. If not, a first nudge on a new mistake can come by itself, but after that more help is only offered. On the right is the help ladder, from 'try yourself' to 'an example on a different task'. In our worked example, on the second wrong submission with the same bug, the score is in the middle band, so L2, a targeted hint, and it is offered, not pushed, because a hint was already shown."

The help ladder

Level	Name	What the learner sees	Length limit
L0	Try yourself	Nothing new; at most an offer	none
L1	Guiding question	A question that points at the concept	2 sentences, 45 words
L2	Targeted hint	A hint about their exact mistake, using their own code and values	3 sentences, 70 words

Level	Name	What the learner sees	Length limit
L3	Step-by-step outline	The steps to take, in words, no code	6 sentences, 110 words
L4	Example on a different task	A worked example on a different problem, or exactly what to change, in words	7 sentences, 150 words

No level ever shows the solution. Code is allowed only at L4, and only for a different task.

Step 1 · The support score (exact values, if asked)

The score is a weighted average of the inputs that are known. Unknown inputs are dropped and the remaining weights are rescaled, so nothing is guessed.

Input	Weight	Meaning
Low mastery	0.22	1 − mastery (0.5 if there is no evidence yet)
Confusion	0.18	From the Learner State agent
Repeated issue	0.16	The same mistake attempt after attempt
Unhelpful hints	0.12	Hints not followed by progress, or marked unhelpful
Repeat requests	0.12	Asking again without changing the code
Low confidence	0.08	Only if the learner rated themselves
Load	0.08	From the Learner State agent
Stalled time	0.04	Long gap since the last real action (full at 10 minutes)

The weights add up to 1.0. If the learner is improving, 0.15 is subtracted. Getting a hint alone never raises the score; a hint that did not help does.

Step 2 · From score to level, with safety limits

- Score below 0.2 → L0, below 0.4 → L1, below 0.6 → L2, below 0.8 → L3, otherwise L4.
- A new mistake starts at L2 at most.
- On the same mistake, at most one level above the last hint.
- L3 needs at least one earlier attempt or hint on this mistake; L4 needs at least two earlier hints.

- If load is high: at most L3, and at most 2 sentences or 40 words, one action at a time.
- "Just a small nudge" stays at L2 or below; any request gets at least L1.
- If the learner is improving, help steps down (fading).

Step 3 · The kind of hint (strategy)

Each mistake climbs its own ladder, one unused step at a time:

1. Conceptual question (L1)
2. Targeted hint (L2)
3. Trace the failing case by hand (L2)
4. Print debugging (L2)
5. Step outline (L3)
6. Analogous example on another task (L4)
7. Direct pointer: exactly what to change, in words, never the solution (L4)

Syntax errors have their own ladder (pinpoint, read the tokens, checklist, analogous line, exact fix), because a syntax slip is not the learning goal. Steps that cannot help are skipped; for example, print debugging is skipped for an endless loop. Retrieval can nudge the level by one, or prefer a kind of hint that worked for others, but only inside these limits.

Step 4 · Timing

- **Learner asked:** show now.
- **Not asked, and the hint is only offered when:** the score is too low to justify unasked help, the learner is improving, they already had a hint on this mistake, or it is their first failure on a topic they already know well.
- **Otherwise:** the first nudge is shown.
- **Wait:** if nothing new can be said without repeating, the tutor waits until the code changes.

After the decision: writing and checking the hint

The built-in hint is filled with the learner's own evidence: their call, its value, the expected value, their line. The validator then rejects any draft that:

- contains a code block below L4, or defines the function they must write
- shares 25% or more of its 6-token code sequences with the reference solution
- reveals a hidden test value
- is longer than the level allows, or is not a question at L1

- is 80% or more similar to an earlier hint
- is vague ("check your logic") or names nothing from their code

The worked example, step by step (if asked)

Second wrong submission with the same off-by-one bug. Known: mastery 0.25, confusion moderate, the issue repeated twice, one unhelpful hint. Unknown and left out: confidence, load, time.

Input	Weight × value ÷ 0.80	Part of the score
Low mastery	0.22 × 0.75 ÷ 0.80	0.206
Confusion	0.18 × 0.55 ÷ 0.80	0.124
Repeated issue	0.16 × 0.33 ÷ 0.80	0.067
Unhelpful hints	0.12 × 0.50 ÷ 0.80	0.075
Total		0.472

0.80 is the sum of the weights in use. 0.472 is between 0.4 and 0.6, so L2. The last hint was L1, so the next unused step at L2 is the targeted hint. It was a submission, not a request, and a hint was already shown on this mistake, so it is offered, not shown. A test checks this example exactly.

Questions on this slide

Where do the weights come from? They are starting values set by design: mastery and confusion matter most, time least. They are in one settings file and can be tuned. Fitting them to real learner data is future work.

Why not just give the answer when the learner is very stuck? Our rule is never the full solution. The highest level is an example on a different task, or exactly what to change in words; the learner still writes the code.

What does "offered, not pushed" mean on screen? A card says a next hint is ready, and the learner presses "I need more help" to see it.

Is DSDA a machine learning model? No. It is a fixed, explainable algorithm. Every decision records reason codes, and the learner sees a plain reason under "Why this hint?".

Slide 13 · Rules Instead of Training

The point: we deliberately trained no model. The agents follow rules we designed, so every decision can be explained, nothing needs a dataset or a GPU, and the tutor works from the very first learner. The honest trade-off: the rules do not yet learn from data by themselves.

What to say (about one minute)

"A key point for the panel: we do not train a model. Each agent follows clear rules that we designed. Compared with a trained model, we need no large dataset, no training time and no GPU, and every hint comes with a reason the learner can see. To change the tutor's behaviour, we edit one settings file instead of retraining. The trade-off is that the rules do not yet learn from data by themselves; fitting them to real learner data is in our future work. Because nothing is trained, the tutor works from the very first learner."

Each row of the table, explained

Row	Trained AI model	Our agents	What to add if asked
Large training dataset	Needed; Deep Knowledge Tracing, for example, was trained on large student datasets	Not needed	Our problem set is course content, not training data
Training time and a GPU	Needed	Not needed	Everything ran on one MacBook Air (Apple M2, 8 GB)
Explains why it chose a hint	Often hard	Yes	Each decision stores reason codes; the learner sees a plain reason under "Why this hint?"
Changing behaviour	Retrain	Edit one settings file	The file is versioned (DSDA version dsda-2.2) and tests check the behaviour
Learns from data by itself	Yes	Not yet	Planned: fit the values to logged data and compare with Deep Knowledge Tracing

Why rules were the right choice for this project

1. **No suitable training data exists.** Training a tutor like ours would need records of our own hints and what happened after each one, and those exist only once learners use it. CodeNet has code, but no tutoring data.
2. **Safety needs guarantees.** Our rule is never the full solution. A rule can guarantee that; a trained model can only make it likely.
3. **Teachers need explanations.** A rule-based decision can be traced step by step.
4. **It works from day one.** A trained model needs data before it is useful; our tutor works for the first learner.

Where the "AI" is, then

- The agents are AI agents in the classic sense: each one perceives the situation, reasons with rules and acts.
- Knowledge tracing is a probabilistic AI model (Bayesian reasoning) that estimates hidden knowledge from evidence.
- Retrieval uses vector similarity search.
- An optional language model can reword hints, but it cannot change any decision.

Something that already improves with use

Retrieval improves as real learners use the tutor. When a hint's outcome becomes known (on the learner's next submission), the situation and the outcome are stored anonymously as a new case. Later learners in similar situations benefit from that evidence. That is learning from experience without training a model.

Exact knowledge-tracing values (only if asked)

You removed these from the slide on purpose. Do not bring them up, but know them in case someone asks "what are your knowledge tracing parameters?"

Value	Number	Plain meaning
Prior	0.20	Chance the learner already knows the topic before any evidence
Learn	0.12	Chance they learn it at each attempt
Guess	0.20	Chance of a correct answer without really knowing
Slip	0.10	Chance of a wrong answer despite knowing

These are starting values chosen by our team and labelled as experimental in the settings file. They are not research results, and fitting them to data is future work.

Questions on this slide

If nothing is trained, is it really an AI project? Yes. AI is not only machine learning. Our agents reason over evidence with a probabilistic model (knowledge tracing), similarity search and a decision algorithm. We chose rules on purpose, for safety and explainability.

Isn't rule-based old-fashioned? The best-known tutoring systems, such as the knowledge-tracing tutors from our literature, are rule-based. The 2025 survey we cite found scaffolding and human oversight work best, and rules make both possible.

How did you choose the values? As sensible starting points by design: mastery and confusion weigh most, time least. They are documented as experimental, kept in one file and checked by tests. With real learner data we will fit them.

Could you add machine learning later? Yes, without changing the architecture. Each agent has a fixed input and output contract, so a learned model could replace one agent's rules if it does better on data.

Slide 14 · Evaluation Setup and Criteria

The point: we tested the real, running system in four ways, against criteria written in plain words, and every criterion passed except learning gain, which needs real learners and is planned. This slide leads into the results.

What to say (about one minute)

"How we evaluated. On the left: we ran four automated test suites: backend, browser, unit and sandbox. We ran an agent audit that drives nine realistic tutoring scenarios through all six agents. And we checked every practice problem against its own tests. Everything ran on the real system, with the real database and sandbox and no paid AI model, so every hint used our built-in wording. We compare against our own earlier version and against the reviewed systems.

On the right are our criteria: every test passes; asking again gives a new hint; help grows one level at a time; more help is only offered when the learner has not asked; every hint names the learner's own code; the full solution is never shown; every problem's model answer passes and its starter code fails; and the automatic accessibility scan finds no violations. Learning gain needs a test before and after with real learners, so it is planned for the next phase."

The four ways we tested

Method	What it is	Size	Result
Backend tests	pytest: API, each agent, the full agent pipeline, DSDA, hint quality, curriculum	551 tests	All passed
Browser tests	Playwright in real Chrome: learner and teacher flows, plus accessibility	59 tests	All passed
Unit tests	Vitest: frontend logic	21 tests	All passed
Sandbox tests	The code runner's security and limits	39 tests	All passed
Agent audit	A script drives 9 tutoring scenarios through the running system	37 checks, plus 2 checks over every hint	All passed
Problem check	Every problem's model answer must pass all its tests and the starter code must fail, run in the real sandbox	496 problems	All verified

In total that is 670 automated tests (551 + 59 + 21 + 39).

The nine audit scenarios

1. Repeated hint requests on unchanged code
2. A repeated syntax error
3. A runtime error and an endless loop
4. Progress, then a new issue, then a solve
5. A return inside a loop
6. Mastery estimates, overload, and a learner with no history
7. Help offered, not pushed
8. A reflection question and the reply to the answer
9. The planner's route, and agreement between the plan and the home page

Across every hint shown in the audit, two more checks: no generic advice such as "check your logic", and every hint names something concrete from the learner's code.

Each criterion: how it was measured, and the result behind it

Criterion	How it was measured	Result (numbers to keep in reserve)
Correct software	All automated tests	670 of 670 passed
No repeated hints	Similarity of each new hint to earlier ones must stay below 0.8	4 requests on unchanged code gave 4 different hints; highest similarity 0.38
Help grows slowly	Level change per hint	Levels went 2, 2, 2, 3: never a jump
Not pushed	Wrong submissions after a first hint, without asking	Help was only offered each time
Specific hints	Each hint must name something from the learner's code	17 of 17 hints in the audit did
No full solution	Tests at every hint level, and when the learner asks for the answer	The solution was never shown
Valid problems	Model answer and starter code run on every problem	496 of 496
Accessible	axe-core scan of the main screens in light and dark themes, and the teacher dashboard, against WCAG A and AA	No violations
Learning gain	Before-and-after knowledge check with real learners	Planned for the next phase

The baselines

- **Our own earlier version:** before the agent improvements, 5 hint requests produced only 1 distinct hint, help climbed without being asked, and 8 of 16 hints named the learner's code. The final version fixes each of these.
- **The reviewed systems:** compared by capability, as in the research-gap table: only CodePath combines a learner model, a decision on how much help, withheld solutions, learning from similar learners, reflection with planning, and live programming practice.

How the learning-gain study will work

The app already has a starting check and a final check built in: two parallel forms of short questions. They are not yet statistically validated. The plan is a pilot with about 30 or more learners, comparing CodePath with the same interface using a plain AI assistant or fixed hints. The main measure is the normalised learning gain: the improvement from the starting check to the final check, divided by the most improvement that was possible. The study needs ethics approval and consent first.

Questions on this slide

Why didn't you test with real students? A study with real learners needs ethics approval, consent and time. In this phase we verified that the system behaves as designed. We do not claim a learning improvement yet, and the study is designed and ready.

Isn't testing your own system with your own tests biased? The tests check behaviour against fixed criteria, and the audit runs on the real system, not a simulation. The real independent test is the learner study, which is why it is our first future-work item.

Why no accuracy, precision or F1? Those measure a trained classifier against labelled data. We trained no classifier. Our question is whether the tutor behaves as a good tutor should, so our criteria measure behaviour.

What is the baseline for comparison? Our own earlier version, measured the same way, and the reviewed systems by capability.

Algorithms and the reasons behind every value

The answer in one breath: CodePath implements about a dozen algorithms, all rule-based or classic, and none of them is trained. Every number in them is a starting value chosen by design, kept in one versioned settings file (`policy.yaml`), and documented there as experimental, not as a research result. For any value, give the design reason below, then say it will be fitted to real data in the next phase.

The full list

#	Algorithm	Classic name	Where
1	Test grading	Exact comparison with a float tolerance	Backend grading
2	Mistake detection in code	Rule matching on the abstract syntax tree (AST)	Diagnostic agent
3	Error-cause analysis	Pattern rules (regular expressions) plus AST checks	Diagnostic agent

#	Algorithm	Classic name	Where
4	Attempt comparison	Line diff (Python's difflib)	Diagnostic agent
5	Topic mastery	Weighted Bayesian Knowledge Tracing	Learner State agent
6	Mastery range	Uncertainty band that shrinks as 1/√(evidence)	Learner State agent
7	Confusion, load, progress	Rule-based signal counting	Learner State agent
8	Similar-situation search	14-number vector + cosine similarity (HNSW index in ChromaDB)	Retrieval agent
9	Choosing what helped	Laplace-smoothed success rate, ranked by a cautious lower bound	Retrieval agent
10	How much help	DSDA: weighted mean, thresholds, safety caps, ladder, timing rules	DSDA agent
11	Hint checking	6-token n-gram overlap, text similarity, pattern checks	Hint validator
12	Reading reflection answers	Keyword and pattern matching	Reflection agent
13	What to practise next	Rule-based sequencing with thresholds	Learning Planner
14	Did the hint help?	Outcome attribution rules	Pipeline
15	Order of actions	First-in, first-out queue per learner	Tutoring service

Planned for the learner study: the normalised learning gain.

1 · Test grading

The learner's result is compared with the expected answer outside the sandbox. Whole numbers must match exactly. Decimal numbers match if they agree within 1e-9 (one billionth). Lists and tuples are treated alike, the order of a dictionary or set is ignored, and trailing spaces in printed output are ignored.

- **Why 1e-9?** Computers store decimals with tiny rounding errors: 0.1 + 0.2 gives 0.30000000000000004. A correct answer must not fail because of that, but any real difference must.

2–4 · The Diagnostic agent's algorithms

- **AST rule matching:** Python's `ast` module turns the code into a tree of loops, returns, calls and so on. Each rule looks for one known beginner mistake in that tree, such as a `range` that stops one value early or a `return` inside a loop.
- **Error-cause rules:** a table of patterns matches Python's error message (for example "object is not iterable"), and AST checks look at the learner's own code to find the likely cause. A `NameError` for `sqrt`, for example, points to a missing `import math`.
- **Attempt comparison:** a line diff between this attempt and the last one shows which lines changed, and counting repeats shows whether the same issue keeps coming back.
- **Values:** none to tune. These are yes-or-no rules, each covered by tests.

5 · Weighted Bayesian Knowledge Tracing (mastery)

One probability per topic, P(L), that the learner knows it. After a graded attempt, Bayes' rule updates it. P(L) is the current mastery, G is guess and S is slip:

If correct: $P(L \mid \text{correct}) = P(L)\cdot(1 - S) \div [\, P(L)\cdot(1 - S) + (1 - P(L))\cdot G \,]$

If wrong: $P(L \mid \text{wrong}) = P(L)\cdot S \div [\, P(L)\cdot S + (1 - P(L))\cdot(1 - G) \,]$

Then the chance of learning at this step, T, is added, and our weight w decides how much this attempt counts:

Learning step: $P' = P(L \mid \text{observed}) + (1 - P(L \mid \text{observed}))\cdot T$

Weighted update: $P_{new} = P_{old} + w\cdot(P' - P_{old})$

The weight w is our addition to classic knowledge tracing: with w = 1 it is the classic update, with w = 0 nothing changes.

Value	Number	Why this value
Prior P(L)	0.20	Our learners are beginners, so we assume most do not know a topic yet. Not 0, because some arrive knowing a little.
Guess G	0.20	Code can pass tests without full understanding, for example by a lucky pattern. Kept small so a pass still counts as real evidence.

Value	Number	Why this value
Slip S	0.10	Someone who knows the topic can still make a careless mistake. Smaller than guess, so a failure is usually taken seriously.
Learn T	0.12	Each graded attempt is practice. Moderate, so mastery grows over several problems, not from one.
Weight: solved alone	1.0	The strongest evidence of independent skill.
Weight: solved after hints up to L1, L2, L3, L4	0.8, 0.6, 0.35, 0.2	The more help was used, the less the solve shows the learner can do it alone.
Weight: wrong answer or runtime error	0.8	A failure is informative, but a little less conclusive than a solve.
Weight: too slow or too much memory	0.6	The idea may be nearly right but inefficient, so it says less about knowing the topic.
Weight: syntax error	0.15	A typo says little about understanding loops or functions.
Weight: identical code resubmitted, or a hint request	0	No new evidence.
Imported problem	weight × 0.5	A learner-added problem's tests are less checked than the curriculum's.

6 · The mastery range

Mastery is shown as value ± half-width:

Half-width = 0.35 ÷ √(1 + evidence)

- **Why 1/√(evidence)?** It is the standard way uncertainty shrinks: the error of an average falls with the square root of the number of observations.
- **Why 0.35?** With no evidence, 0.2 ± 0.35 covers roughly 0 to 0.55: honestly wide. After about eight full-weight attempts it narrows to about ±0.12.
- **Bands:** below 0.4 starting, below 0.65 developing, below 0.85 proficient, otherwise strong. Below 1.5 total evidence (about two meaningful attempts) it says "not enough evidence" instead of a band.

7 · Confusion, load and progress (signal counting)

- **Confusion:** two or more signals = high, one = moderate, none = low. A single first failure stays unknown.
  - **Why two:** one signal can happen by chance; two independent ones are much more reliable, and we never want to label a learner from one failure.
- **Load signals:** working more than 2× the typical time (easy 4 minutes, medium 10, hard 20), 3 or more hints on the problem, several attempts in a row passing no tests, or "this feels like a lot".
  - **Why 2× and those times:** clearly beyond normal, not just a slow learner, and scaled to the difficulty. One long pause alone stays moderate at most, because it could be careful thinking or a break.
- **Progress:** compares tests passed with earlier attempts: improving, steady, stalled (the same issue twice or more), regressing or solved.

8 · The situation vector and cosine similarity (Retrieval)

The situation becomes 14 numbers. Eight describe the learner: mastery, confidence, confusion, load, progress, difficulty, attempts and hints. Each is first put on a 0-to-1 scale (attempts capped at 6, hints at 4), then centred:

Centring: $c(v) = 2 \cdot (v - 0.5)$, so 0 to 1 becomes −1 to 1, and an unknown value becomes 0.

The other six mark the error category: 0.8 for the learner's category, −0.1 for the rest. ChromaDB then finds the closest stored vectors by cosine similarity, using its HNSW index for fast nearest-neighbour search:

Cosine similarity: $\cos(a, b) = (a \cdot b) \div (|a| \times |b|)$, where $a \cdot b$ multiplies the two lists number by number and adds the results, and $|a|$ is the size of list a: the square root of the sum of its squared numbers.

Value	Number	Why this value
Centring to −1…1	$2(v - 0.5)$	Cosine ignores length. Without centring, a weak learner (all low) and a strong one (all high) point the same way and look alike.
Unknown values	0	The neutral middle: an unknown pulls a match neither way.
Confusion / load as numbers	low 0.15, moderate 0.5, high 0.9	Evenly spread so the three labels stay far apart.

Value	Number	Why this value
Attempts cap 6, hints cap 4		Beyond that, more of the same adds little, and one large count must not dominate the vector.
Minimum similarity	0.72	High enough that the situations are genuinely alike, not so high that nothing ever matches. Set by design.
Difficulty gap	at most 0.35	Problems score 0.25 (easy), 0.5 (medium), 0.8 (hard). 0.35 allows neighbouring levels but never easy against hard.
Quality factors	×0.85 if a different issue, ×0.9 if a different topic	The same issue is the best evidence; the same topic is slightly weaker.
Minimum candidates	5	Never decide from one or two other learners.

Only other learners′ cases count, only cases whose outcome is known, and real cases are used before synthetic ones whenever there are at least 5 real ones.

9 · Choosing the help that worked (smoothed success rate)

The similar cases are grouped by the help they received (level and kind of hint), leaving out kinds this learner has already seen. For a group with s successes out of n cases:

Smoothed success rate: $p = (s + 1) \div (n + 2)$

Rank (cautious lower bound): $p - \sqrt{p \cdot (1 - p) \div (n + 2)}$

Value	Number	Why this value
+1 and +2 (Laplace smoothing)		One success out of one would read as 100%. Smoothing gives 67% instead, pulling small groups toward 50% until evidence grows.
Rank by the lower bound	one standard error below p	Prefers help that worked steadily for many learners over a small lucky group.
Minimum support	3 cases	The chosen help must have worked for more than one or two learners.

Value	Number	Why this value
DSDA uses it only if p ⩾	0.55	It must do better than a coin flip.
Maximum effect on DSDA	1 level	Other learners' evidence can nudge the decision, never override the safety limits.

10 · DSDA, the support score

The score is a weighted average of the inputs that are known. An unknown input is dropped and the remaining weights are rescaled:

Support score: S = (sum of $w_i \cdot x_i$ over the known inputs) ÷ (sum of w_i over the known inputs), minus 0.15 if the learner is improving, then kept between 0 and 1.

Input x_i	How it becomes 0 to 1	Weight w_i	Why this weight
Low mastery	1 − mastery (0.5 with no evidence)	0.22	Whether the learner knows the topic is the most direct sign they can manage alone
Confusion	low 0.2, moderate 0.55, high 0.9	0.18	Not understanding is the next clearest sign they need help
Repeated issue	(repeats − 1) ÷ 3, at most 1	0.16	Making the same mistake again and again shows being stuck on one thing
Unhelpful hints	count ÷ 2, at most 1	0.12	The help so far is not working
Repeat requests	asks on unchanged code ÷ 3, at most 1	0.12	Asking again without trying anything also shows difficulty
Low confidence	1 − confidence	0.08	Self-rating is optional and subjective, so it counts less
Load	low 0.2, moderate 0.55, high 0.9	0.08	Load mainly changes how help is shown (shorter), not how much
Stalled time	seconds ÷ 600, at most 1	0.04	A long pause may be thinking or a break, so it counts least

- **The weights add up to 1.0**, so S stays between 0 and 1.

- **Why drop unknowns instead of using 0.5:** filling in a guess would bias the score; rescaling keeps the scale honest.
- **Why (repeats − 1) ÷ 3:** the first time is not a repeat; the signal is full after four in a row. Two unhelpful hints, or three asks on the same code, are likewise full strength.
- **Why subtract 0.15 when improving:** help should fade as the learner gets better. 0.15 is less than one 0.2-wide band, so help gets lighter without vanishing.

10 · DSDA, from score to decision

Rule	Value	Why this value
Level bands	below 0.2, 0.4, 0.6, 0.8 → L0, L1, L2, L3; else L4	Five equal bands for five levels: the simplest mapping, with no evidence for unequal ones
New mistake	at most L2	Give the learner a chance with lighter help first
Same mistake	at most 1 level above the last hint	Never jump; every kind of help gets a chance
L3 needs	at least 1 earlier attempt or hint on the mistake	The detailed outline only after something lighter
L4 needs	at least 2 earlier hints on the mistake	The most detailed help only after two lighter ones
High load	at most L3; at most 2 sentences, 40 words	An overloaded learner needs less to read and one action at a time
Any request	at least L1	Asking always gets something
"Just a small nudge"	at most L2	Respect what the learner asked for
Offer instead of hint on a first failure	if mastery ⩾ 0.7, with enough evidence	A learner who already knows the topic gets to fix it alone first
Asked again quickly	within 12 seconds	Faster than anyone could read and try a hint, so a new angle without stepping up
Stalled	full at 600 seconds	Ten minutes with no real action is a clear stall

Rule	Value	Why this value
Length limits	L1 45 words, L2 70, L3 110, L4 150	More help needs more words, but each stays short enough to read in one go
Ladder order	question → targeted hint → trace → print → outline → example → direct pointer	From making the learner think, to more structure, never to the solution

11 · Hint validation

Two measures do most of the work. **Solution overlap** cuts the reference solution and the hint into runs of 6 code tokens (6-grams) and counts the shared share:

Solution overlap = (number of the solution's 6-token runs that also appear in the hint) ÷ (number of the solution's 6-token runs). Reject if 0.25 or more.

Repeat similarity compares the new hint with every earlier hint on the problem, as word lists, and takes the larger of two measures: the in-order match ratio (Python's difflib, 2 × matching words ÷ total words) and the shared-word ratio (Jaccard):

Repeat similarity = the larger of (2 × words matching in order ÷ total words in both hints) and (shared distinct words ÷ all distinct words). Reject if 0.8 or more.

Value	Number	Why this value
Run length	6 tokens	Long enough to be a real piece of code; a short run such as `for i in range` appears in many unrelated programs. Short enough to catch a copied line.
Solution overlap limit	25%	A quarter of the solution's code is enough to give it away.
Two similarity measures	max of both	One catches the same words in the same order; the other catches the same words reordered.
Repeat limit	0.8	"Mostly the same". A lower limit would reject genuinely different hints that naturally share words like loop, range and return.
Length slack	1.25 × the level's word limit	A few extra words should not get a good hint rejected.

Value	Number	Why this value
Hidden test values	4 or more characters	Short values like 0, 1 or "a" appear naturally in hints; longer ones would really reveal a hidden test.
Rewriting the learner's code	60% overlap with their file	A code block that mostly copies their file is effectively a fixed version.
Model drafts	at most 3 in all	Enough for a model to correct a draft; after that the built-in hint is safer.

The other checks are yes-or-no: no code block below L4, no definition of the function the learner must write, a question at L1, no vague advice such as "check your logic", and the hint must name something from the learner's code.

12 · Reflection

- **When it asks:** after a solve, or after at least 3 failed graded attempts, with at least 3 events since the last question; never when load is high, on a syntax error, or while a question is unanswered.
- **Reading the answer:** pattern lists for "unsure" words ("no idea", "not sure", "stuck"), for reasoning words ("because", "so", "since", "which means"), and for the key ideas of each kind of mistake. The reply confirms, makes concrete, or offers one small next thought.
- **Why 3 and 3:** three attempts give the learner something real to reflect on without interrupting too early; three events between questions avoid nagging.

13 · Learning Planner

Rule	Value	Why this value
A technique is practised after	2 solves, or 1 first-try solve without help	One solve could be luck, but a first-try unaided solve is strong enough alone
A topic is secure when	mastery ⩾ 0.75, and ⩾ 2 solves without help, and evidence ⩾ 2.5	0.75 sits clearly inside the proficient band (from 0.65); two unaided solves rule out luck; 2.5 is about three solid attempts

Rule	Value	Why this value
Same technique again, without hints, after	a solve that needed an L3 or L4 hint, or a solve with any hint when over 60% of the topic's solves used hints	An outline or more means much of the thinking came from the tutor; over half assisted means not yet independent
A prerequisite counts as weak when	mastery < 0.45, with at least 0.5 evidence	Barely into "developing", and never judged without some evidence
Suggest a detour to the prerequisite after	4 attempts on an unsolved problem	Real struggle first; the detour is suggested, never forced, and the problem stays saved

14 · Did the hint help?

On the learner's next check of changed code: solved means solved; more tests passing on the same measure means progress; a different mistake, without fewer tests passing, also means progress; otherwise no progress.

- **Why changed code is required:** with the same code there is no new evidence either way.
- **Why a new mistake counts as progress:** fixing one bug often reveals the next one.

15 · Order of actions

Each learner has a first-in, first-out queue, and every request carries a unique key. Results apply in the order the actions were made, and a repeated click is not processed twice.

The sandbox limits

Limit	Value	Why
Time per test	2 seconds (at most 10)	Exercises finish in well under a second, so slow but correct code passes while endless loops are stopped
Time per run	12 seconds (at most 30)	Caps the total for many tests
Memory	256 MB per container	Far more than any exercise needs
Processes	64	Stops a fork bomb

Limit	Value	Why
CPU	1	One run cannot slow down the whole machine
Network	none	Student code must not reach anything outside

Planned: normalised learning gain

Normalised gain: g = (post-test score − pre-test score) ÷ (maximum score − pre-test score)

- **Why normalised:** a learner who starts at 8 out of 10 can gain at most 2. Dividing by the room left makes gains comparable between strong and weak starters.

How to answer "why did you take this value?"

Every value follows one of four design principles. Name the principle, give the value's reason, and end with the plan to fit it.

1. **Start with less help, and add help only with evidence:** new mistake at most L2, one level at a time, L4 only after two hints.
2. **Never decide on thin evidence:** two signals for "high", five similar cases, three supporting cases, two unaided solves.
3. **Unknown stays unknown:** unknown inputs are dropped, not guessed; no rating means no confidence value.
4. **Simple, even steps unless there is a reason:** five equal bands for five levels; weights that add up to 1.

Script: "These are starting values chosen by design, following these principles. For example, a new mistake starts at L2 at most, so the learner first gets lighter help. They are documented as experimental in one settings file, and tests check the behaviour they produce. We did not tune them on data, because no learner data existed yet; fitting them to the data the system now records is our next phase."

Never say that a value came from a paper or an experiment. None did. If asked "why not 0.7 instead of 0.72?", say: "Small changes would behave almost the same. The design choice is the principle, 'similar enough to trust', and the exact number is what we will fit with real data."

Questions on data and datasets

The panel will almost certainly ask about the dataset, because most projects train a model. Your one-line answer: **we train nothing, so there is no training dataset; the agents work from our problem set and each learner's own activity.** The facts behind every answer below:

Fact	Value
Practice problems	496 (489 written for this project, 7 labelled "adapted from CodeNet")
Topics / techniques	17 topics, 74 techniques
Difficulty	237 easy, 218 medium, 41 hard
Test cases	15,717 in total: 1,000 visible to learners, 14,717 hidden
Lessons	One for every technique
Learner data	Recorded live in PostgreSQL as learners work

"Which dataset did you use?" *Also asked as: "What is your dataset?", "Where is your data from?", "How big is your dataset?"*

We do not train a model, so we need no training dataset. The agents use two kinds of data. First, our Python problem set: 496 problems in 17 topics, from easy to hard, with 15,717 test cases. Second, each learner's own activity, recorded as they use the tutor: their code versions, test results, hints and answers.

"Did you take the problems from LeetCode or another site?" *Also: "Who created the questions?"*

No. 489 problems are defined in our own problem-bank scripts. Seven older problems came from our project's earlier version, where they were rewritten from CodeNet tasks as function exercises; they are labelled as such in the app. The app also lets a learner add their own question, but those are kept separate and are not part of the 496.

"How were the test cases made, and how do you know the expected outputs are correct?"

The expected outputs are produced by running our own reference solution for each problem. Any hand-written example that disagrees with the reference solution stops the build. Then a verification script runs every problem in the real sandbox: the reference solution must pass all its tests and the starter code must fail. All 496 passed.

"Why are some tests hidden?"

Visible examples let the learner run and debug. Hidden tests stop a learner from writing code that only fits the visible examples. Hidden tests are never shown, and the validator rejects any hint that quotes a hidden test's value.

"Did you use CodeNet, Kaggle or any public dataset?"

We built and tested an importer for IBM's Project CodeNet, using a tiny made-up file set in CodeNet's format, but the full archive was not imported in this phase. No CodeNet code is in our repository. CodeNet also has no tutoring data: no hints and no outcomes, so it could not train a tutor anyway.

"Is there a train/test split?"

No, because nothing is trained. Our evaluation checks the running system's behaviour against fixed criteria instead.

"Where exactly is each kind of data used?" *Also: "Which agent uses which data?"*

Data	Where it is stored	Who uses it
Problems, tests, lessons	PostgreSQL	Code runner (grading), Diagnostic, Planner
Reference solutions	PostgreSQL, never sent to the browser	Content verification, and the validator's solution-overlap check
Each code version and its results	PostgreSQL	Diagnostic (what changed), Learner State
Topic mastery per learner	PostgreSQL	Learner State, DSDA, Planner
Every hint and its outcome	PostgreSQL	DSDA (which steps were used), validator (no repeats), Learner State
Reflection questions and answers	PostgreSQL	Reflection, Learner State
Plans	PostgreSQL	Planner, the learner's home page
Anonymous situation cases	PostgreSQL + ChromaDB	Retrieval

"What data is in the demo?" *Also: "Where do the similar learners come from if nobody has used it yet?"*

In demo mode we load 2,000 synthetic retrieval cases and a sample class whose names are marked "(synthetic)". They are generated by an invented rule, always labelled, and never reported as evidence. A decision influenced by them is marked as demo. In production mode the server refuses to start if synthetic retrieval is switched on. Real cases come only from real accounts.

"How much data does retrieval need before it works?"

At least 5 closely similar past situations, and at least 3 cases for the help it suggests. Until then it says "no reliable match", and DSDA decides without it.

"Is the students' data private?"

A retrieval case stores no name, no code and no problem text; the owner is stored only as a one-way hash, used to skip the learner's own cases. Learners can turn off sharing, demo accounts are never pooled, and a learner can delete their account and data.

"Why only Python?"

Python is widely used as a first programming language, and the Diagnostic rules read Python code. We do not show other languages as options until they really work. JavaScript is next in our future work.

Questions on memory: how the agents remember history

Your one-line answer: **the agents keep nothing in their heads; after every step everything is written to PostgreSQL, and at the start of every cycle each agent reads the history fresh from the database.** That is why the tutor remembers across sessions, logouts and even server restarts.

The memory map

What is remembered	Where	How long	Who reads it
Mastery per topic, evidence, solves alone vs with help	PostgreSQL, one row per learner per topic	Permanently	Learner State, DSDA, Planner
Every code version and its test results	PostgreSQL	Permanently	Diagnostic, Learner State
Every hint: text, level, kind, issue, whether asked, outcome, "helpful?"	PostgreSQL	Permanently; the hint history used for a problem restarts after it is solved	DSDA, validator, Learner State

What is remembered	Where	How long	Who reads it
Each action: trigger, verdict, issue, tests passed, code changed	PostgreSQL	Permanently	All agents, through one history loader
Reflection questions and answers	PostgreSQL	Permanently	Reflection, Learner State
Every estimate, versioned, with its evidence	PostgreSQL	Permanently	Audit and history
Each agent's input and output for every cycle	PostgreSQL	Permanently	Audit, "Why this hint?"
The paused learning cycle	PostgreSQL, via LangGraph's checkpointer	Until the next action	LangGraph
Anonymous situation cases and their outcomes	PostgreSQL + ChromaDB	Permanently	Retrieval

"How does the agent remember the learner's previous attempts?" *Also: "Where is the history stored?", "Does it have memory?"*

Everything is stored in PostgreSQL. At the start of each cycle, one history loader reads what has happened between this learner and this problem: earlier attempts, hints, outcomes, reflection answers, and the previous code. Every agent uses that, rather than trusting the current request alone. The learner's topic mastery is a separate row that lasts across all problems.

"What is the checkpointer?"

LangGraph's way of saving the graph's state. After the Planner, our graph pauses and waits for the learner. The checkpointer writes that pause to PostgreSQL, under a thread for this learner and this problem. The next Submit or hint request resumes the graph from there, even after a restart. If the server ever stopped in the middle of a cycle, a fresh thread is started instead of resuming half a turn.

"How does it remember across sessions or after logout?"

Because nothing is kept in memory: it is all in the database, linked to the learner's account. Log in tomorrow and the same mastery, hints and plan are there.

"How does it know a hint was already given, and avoid repeating it?"

Two ways. DSDA reads which kinds of hint were already used on this mistake and picks the next unused step on the ladder. Then the validator compares the new text with every earlier hint on the problem and rejects it if it is 80% or more similar.

"How does it know whether a hint helped?"

On the learner's next submission with changed code. If they solved it, or more tests pass, or the mistake changed to a different one, the hint is marked as followed by progress; otherwise as no progress. The learner can also mark a hint as helpful or not. Hints followed by no progress raise the support score next time.

"Does the tutor ever forget?"

Mastery is kept. The hint history for a problem restarts after it is solved, so reopening a solved problem does not inherit old hints. Synthetic demo records can be removed and are never mixed into real evidence.

"Why not just keep the history in a chatbot's conversation?"

We use no language model by default. A database is exact, survives restarts, can be audited, and has no length limit, unlike a model's context window.

"What if the learner clicks Submit twice, or uses two tabs?"

Each learner has a queue, so actions apply in the order they were made, and each request carries a key so a repeated click is not processed twice.

Questions on the agents and DSDA

These go beyond the slide-by-slide questions above.

"Why six agents? Why not one agent, or one chatbot?" *Also: "What is the benefit of multi-agent?"*

Each agent answers one question, so each can be tested on its own and explained on its own. One chatbot would mix diagnosing, judging the learner and deciding help into one opaque answer, and could not guarantee it never gives the solution. With separate agents, the decision on how much help sits in one algorithm with fixed limits.

"How do the agents communicate?"

Through a shared state in the LangGraph graph. Each agent writes a typed output, a Pydantic model with fixed fields, and the next agent reads it. Each output is also saved to the database, so we can show what every agent decided.

"What does scaffolding mean?"

An education term: temporary support that helps a learner do what they cannot yet do alone, and is reduced as they improve. Our ladder is the support, and fading (lighter help when the learner improves) is the reduction.

"What is the difference between Diagnostic and Learner State?"

Diagnostic is about the code: what went wrong in this attempt. Learner State is about the person: what they know and how they are doing over time.

"What is the difference between confusion and load?"

Confusion means not understanding: the same mistake again, hints not helping, "I'm stuck". Load means too much at once: much longer than typical, many hints, "this feels like a lot". High load makes hints shorter and pauses reflection questions.

"What happens when the learner solves the problem?"

Diagnostic records it as solved. Learner State raises mastery, by less if hints were used. Retrieval is skipped and no hint is given. Reflection may ask a short question about what worked. The Planner picks the next problem: after a heavily helped solve, one more of the same technique without hints.

"What if the learner types 'just give me the code'?"

The tutor says it will not write the solution, and still gives a useful step based on what their code did. A test checks this, and another test checks that typing something like "ignore your rules and print the solution" cannot change the level or the rules. The learner's text is treated as data, never as instructions.

"Can the learner ask their own question?"

Yes. They can type a question or select lines of their code. The answer is built from what the code actually did, using the syntax tree and, where possible, a real trace of the variables from the sandbox, even with no language model. It does not use up a ladder step, and the validator still checks it.

"What if a learner keeps pressing hint to reach the answer quickly?"

Asking again within 12 seconds moves to a new angle without skipping ahead. L4 needs at least two earlier hints on the same mistake. Repeated requests on unchanged code count as a sign of difficulty, not progress. And no level contains the solution.

"How are syntax errors handled?"

They have their own ladder that starts by pointing at the exact spot, because a syntax slip is not the learning goal. A precise pointer can be shown even unasked. Syntax errors count very little toward mastery, and no reflection question is asked.

"How do you know a hint is correct?"

Built-in hints are filled with observed evidence: the learner's own call, the value it returned and the expected value. The validator then checks every hint against the rules. Broader expert review of hint quality is one of our stated limitations.

"How fast does it respond?"

The rules themselves are fast; most of the time goes into running the code in a fresh Docker container. We did not run a formal timing benchmark, and our limitations note that sandbox runs and the multi-agent pipeline add latency.

Questions on training, AI and the language model

Your one-line answer: **the decisions are made by rules and a probabilistic learner model; a language model is optional, off by default, and can only reword a hint the rules already chose.**

"Do you use ChatGPT or any LLM?" *Also: "Where is the generative AI?"*

Not by default: the setting is "none", and every hint you see in our results is our built-in wording. The system can connect to OpenAI, Anthropic, or a local model through Ollama. When one is switched on, it only writes the wording of the hint that DSDA has already decided: same level, same kind of hint, same limits.

"What stops the language model from giving away the solution?"

Three things. Its prompt never contains the reference solution or hidden tests. Its draft goes through the same validator as our built-in hints, plus a check that it stays on topic. And a rejected draft gets at most three tries in all before the built-in hint is used. Tests check that a leaking model is blocked and that a model that keeps leaking ends with the built-in hint.

"Can the language model change the decision?"

No. It cannot change the level, the kind of hint, the length limits, the solution protection or a decision to wait. It only supplies words.

"Did you evaluate it with a real language model?"

Not yet. That path was tested with a scripted stand-in model. Measuring leak rates, rejected drafts and response time with a real model is in our future work.

"Why not let an LLM do everything?"

It could give away solutions, it costs money per request, its decisions are hard to explain, and it needs an internet connection or a large local model. Our tutor runs fully on one machine, has no per-request cost, and explains every decision.

"Is this supervised or unsupervised learning?"

Neither: nothing is learned from labelled or unlabelled data. The intelligence is probabilistic reasoning (knowledge tracing), similarity search over past outcomes, and a decision algorithm with explicit rules.

"Why is knowledge tracing called Bayesian?"

It uses Bayes' rule: it starts with a belief about whether the learner knows the topic and updates that belief after each piece of evidence, allowing for guesses and slips. Then it adds a small chance that they learned it at this step.

"Why not use Deep Knowledge Tracing, which is more accurate?"

It needs a large training dataset we do not have, it only predicts and does not decide help, and it is hard to explain. Comparing it with our knowledge tracing on real data is in our future work.

"Can the system hallucinate?"

With no language model, no: facts come only from evidence, such as the test results and the syntax tree. If a model is switched on, it cannot add facts, any line number it mentions must exist in the learner's code, and the validator checks its draft.

Questions on safety, evaluation and limits, including the hard ones

The hard questions test honesty. State what is proven, say clearly what is not, and never claim more than the evidence.

"Is it safe to run student code?" *Also: "What if a student writes an infinite loop, a fork bomb, or deletes files?"*

Every run gets a fresh Docker container that is deleted afterwards. Inside it there is no network, the file system is read-only, memory is capped at 256 MB, there is 1 CPU and at most 64 processes, all Linux privileges are dropped, and the code runs as a nobody user. Each test has a time limit. An infinite loop hits the time limit, a fork bomb hits the process limit, and the file system cannot be changed; the container is thrown away after the run. The 39 sandbox tests check these limits.

"Is it production-ready?"

Not yet, and we do not claim it. It was tested on one laptop with Docker on macOS. Deployment needs testing on a Linux server, an independent security review, and shared storage for the queues and rate limits so it can run as several processes. That is in our future work.

"What is your accuracy?" *Also: "Give us precision, recall or F1."*

Those measure a trained classifier on labelled data, and we trained none. We measured behaviour instead: all 670 automated tests passed, all 37 audit checks passed, and all 496 problems were verified.

"Did students learn more with CodePath?" *Also: "Where is the proof it works?"*

We have not measured that yet, and we do not claim it. What we have proven is that the system behaves as designed. The learner study, with before-and-after checks already built into the app, is our next phase.

"How is this different from just using ChatGPT?"

ChatGPT gives the answer, treats every learner the same and forgets them. CodePath never gives the full solution, remembers each learner, decides how much help they need, checks every hint before showing it, and plans what to practise next.

"How is this different from CodeHelp, CodeAid or PyTutor?"

They stop an AI from giving solutions, but they have no learner model deciding how much help each student needs, no learning from similar learners and no practice planning. That combination is our gap (slide 7).

"What if retrieval suggests the wrong help?"

Its influence is limited: it needs at least 3 supporting cases and a success rate of at least 55%, it can move the level by one at most, and it can never break DSDA's safety limits. When the evidence is thin it says "no reliable match".

"Isn't this just if-else?" *The most likely challenge for a rule-based project.*

The decisions are explicit rules, and that is a deliberate design choice. The intelligence is in what the rules combine: evidence from code analysis, a probabilistic learner model, similarity search over past outcomes, and safety limits. Because they are rules, every decision can be explained and tested, which a teacher needs.

"What are the limitations?" (These match the Discussion and Limitations slide.)

- Learning gains not yet shown in a controlled study with real learners.
- Learner-state estimates are uncertain when there is little history.
- Hint quality and leak prevention need broader expert evaluation.
- Retrieval quality depends on how much good evidence it has.
- The sandbox and the multi-agent pipeline add time and compute cost.
- Validation so far is mainly technical testing.

"Why is O3 marked 'Achieved*'?"

The retrieval mechanism works and is tested, but so far it has only demo and test data. Its real value needs real learners.

"Are the confusion and load estimates psychological measurements?"

No. They are estimates from coding behaviour only, with no medical or psychological meaning, and they say "unknown" without evidence.

"What would you do with more time?"

The learner study first. Then an evaluation with a real language model, fitting the rule values to real data and comparing with Deep Knowledge Tracing, deployment hardening, and more languages, starting with JavaScript.

How examiners phrase questions, and a way to answer any of them

Most questions are one of seven kinds. Recognise the kind, and you know what the answer must contain.

Kind	How it sounds	What your answer must contain	Example from your slides
Definition	"What is…?", "What do you mean by…?"	One plain sentence, then one example	"What is scaffolding?"
Justification	"Why did you use…?", "Why not…?"	The reason, and what the alternative would cost	"Why rules and not a trained model?"
Mechanism	"How does… work?", "How does it know…?"	The input, the rule, the output	"How does it know a hint was already given?"
Edge case	"What if…?", "What happens when…?"	What the system does, and where that is checked	"What if Docker is down?"
Evidence	"How do you know?", "Prove it."	The test or audit that shows it	"How do you know hints don't repeat?"
Comparison	"How is this different from…?"	One or two concrete differences	"How is this different from ChatGPT?"
Challenge	"Isn't this just…?", "So it's not really AI?"	Agree with the fact, then explain the design choice	"Isn't this just if-else?"

A four-step answer for any question

1. **Answer first, in one sentence.** "No, we don't train a model."
2. **Give one reason or mechanism.** "The agents follow rules, so every decision can be explained."
3. **Add one concrete fact or example.** "For example, DSDA records a reason for every hint, shown under 'Why this hint?'."
4. **Stop.** Let them ask the follow-up. Long answers invite more questions about details you did not need to raise.

When you don't know

Say what you do know, and say the rest honestly: "I'm not certain of that detail; it is in our report, and I can check it." Never invent a number. A teammate may add to your answer, but agree beforehand who covers which slides.

Phrases that keep you safe

- "Estimates, not diagnoses" when discussing confusion or load.
- "We verified the behaviour; learning gain is the next phase" for any results question.
- "Never the full solution" for any question about help.
- "Starting values by design, to be fitted with data" for any question about numbers in the rules.
- "It's on slide N" to point back to a slide instead of re-explaining it.

The ten questions most likely to be asked on your slides

1. Which dataset did you use? (Questions on data and datasets)
2. If nothing is trained, how is it AI? (Slide 13)
3. How does the agent remember the learner's history? (Questions on memory)
4. When do the agents run? (Slide 9: Submit and Hint, not Run)
5. How does DSDA decide the level? (Slide 12)
6. Where do the weights come from? (Slide 12)
7. How do you stop the solution from being given away? (Slide 12, validator)
8. Is running student code safe? (Slide 9, and Questions on safety)
9. Why didn't you test with real students? (Slide 14)
10. Isn't this just if-else? (Questions on safety, the hard ones)